

Advanced Computer Systems Project 2 Report

Ivan Cheng (ECSE)

Rensselaer Polytechnic Institute

ECSE 4320 Advanced Computer Systems

Dr. Tong. Zhang

December 15th, 2025

Overview

Dense matrix multiplication (GEMM) is a building block in numerical computing, deep learning, and high performance systems. When matrices become sparse, computing full GEMM is wasteful in both computation and memory bandwidth. Sparse dense SpMM, uses compressed sparse row format which reduces both arithmetic operations and memory traffic especially at low densities.

Performance for dense vs sparse matrix multiplications depends on factors such as:

- SIMD vectorization
- Cache aware blocking
- multithreading
- Working set size
- sparsity patterns and density
- arithmetic intensity
- memory bandwidth bottlenecks

This project implements and evaluates:

- Scalar dense GEMM
- SIMD blocked GEMM
- Multithreaded blocked GEMM
- CSR based SpMM (single threaded & multithreaded)

Methodology

Dense Matrix Generation

Dense matrices were generated with random values $\in [-1, 1]$.

Sparse CSR Matrix Generation

Sparse matrices were generated in compressed sparse row format with user selected density p and uniform random non zero distribution.

Dense GEMM Implementations

- Scalar GEMM: Classical triple loop reference.
- Blocked SIMD GEMM: Cache aware $i/j/k$ tiling with Advanced Vector Extensions
- Multithreaded GEMM: OpenMP parallelization across tiles.

Sparse Dense SpMM Kernel

- Compressed Sparse Row Format: Sparse matrix stored using row pointers, column indices, and values arrays.
- Single thread SpMM: Processes only nonzero entries of matrices to compute sparse dense multiplication on one core.
- Multithreaded SpMM: Splits compressed sparse rows across threads to parallelize sparse dense multiplication.

Benchmarking Method

- FLOP Counting: Computes total operations from matrix dimensions to convert time into GFLOP/s.
- Time Measurement: Records elapsed time for each run
- Repetitions: Runs each experiment multiple times and keeps the minimum runtime.

Machine Model

- Intel i7-11850H
- Base clock: 2.5 GHz
- AVX-512 available
- Theoretical FP32 peak: 640 GFLOP/S
- DDR4-3200 peak bandwidth $\approx$ 51.2 GB/s
- Environment: WSL2 Linux subsystem

Commands that were used in experiment:

```
g++ -O3 -march=native -fopenmp -linclude src/matrix_gen.cpp src/gemm.cpp src/spmm.cpp  
src/bench_gemm_spmm.cpp -o bench_gemm_spmm  
./bench_gemm_spmm 64 64 64 0.1 check 1 1  
./bench_gemm_spmm 1024 1024 1024 0 dense_scalar 1 3  
./bench_gemm_spmm 1024 1024 1024 0 dense_simd 1 3  
./bench_gemm_spmm 1024 1024 1024 0 dense 4 3  
./bench_gemm_spmm 1024 1024 1024 0.05 dense 1 3  
./bench_gemm_spmm 1024 1024 1024 0.05 dense 2 3  
./bench_gemm_spmm 1024 1024 1024 0.05 dense 4 3  
./bench_gemm_spmm 1024 1024 1024 0.05 dense 8 3  
./bench_gemm_spmm 1024 1024 1024 0.05 spmm 1 3  
./bench_gemm_spmm 1024 1024 1024 0.05 spmm 2 3  
./bench_gemm_spmm 1024 1024 1024 0.05 spmm 4 3  
./bench_gemm_spmm 1024 1024 1024 0.05 spmm 8 3  
./bench_gemm_spmm 128 128 128 0.05 dense 4 3  
./bench_gemm_spmm 256 256 256 0.05 dense 4 3  
./bench_gemm_spmm 512 512 512 0.05 dense 4 3  
./bench_gemm_spmm 1024 1024 1024 0.05 dense 4 3
```

```

./bench_gemm_spm 1536 1536 1536 0.05 dense 4 3
./bench_gemm_spm 2048 2048 2048 0.05 dense 4 3
./bench_gemm_spm 3072 3072 3072 0.05 dense 4 3
./bench_gemm_spm 128 128 128 0.05 spmm 4 3
./bench_gemm_spm 256 256 256 0.05 spmm 4 3
./bench_gemm_spm 512 512 512 0.05 spmm 4 3
./bench_gemm_spm 1024 1024 1024 0.05 spmm 4 3
./bench_gemm_spm 1536 1536 1536 0.05 spmm 4 3
./bench_gemm_spm 2048 2048 2048 0.05 spmm 4 3
./bench_gemm_spm 3072 3072 3072 0.05 spmm 4 3
./bench_gemm_spm 2048 2048 2048 0.0005 dense 4 3
./bench_gemm_spm 2048 2048 2048 0.001 dense 4 3
./bench_gemm_spm 2048 2048 2048 0.002 dense 4 3
./bench_gemm_spm 2048 2048 2048 0.005 dense 4 3
./bench_gemm_spm 2048 2048 2048 0.01 dense 4 3
./bench_gemm_spm 2048 2048 2048 0.02 dense 4 3
./bench_gemm_spm 2048 2048 2048 0.05 dense 4 3
./bench_gemm_spm 2048 2048 2048 0.1 dense 4 3
./bench_gemm_spm 2048 2048 2048 0.2 dense 4 3
./bench_gemm_spm 2048 2048 2048 0.5 dense 4 3
./bench_gemm_spm 2048 2048 2048 0.0005 spmm 4 3
./bench_gemm_spm 2048 2048 2048 0.001 spmm 4 3
./bench_gemm_spm 2048 2048 2048 0.002 spmm 4 3
./bench_gemm_spm 2048 2048 2048 0.005 spmm 4 3
./bench_gemm_spm 2048 2048 2048 0.01 spmm 4 3
./bench_gemm_spm 2048 2048 2048 0.02 spmm 4 3
./bench_gemm_spm 2048 2048 2048 0.05 spmm 4 3
./bench_gemm_spm 2048 2048 2048 0.1 spmm 4 3
./bench_gemm_spm 2048 2048 2048 0.2 spmm 4 3
./bench_gemm_spm 2048 2048 2048 0.5 spmm 4 3
./bench_gemm_spm 1024 1024 1024 0 dense 4 3
./bench_gemm_spm 1024 1024 1024 0.05 spmm 4 3
./bench_gemm_spm 2048 2048 2048 0 dense 4 3
./bench_gemm_spm 2048 2048 2048 0.05 spmm 4 3

```

Correctness Verification

./bench_gemm_spm 64 64 64 0.1 check 1 1 was used to evaluate correctness

Max abs error (blocked_omp vs naive) = 0

Max abs error (SpMM vs naive) = 0

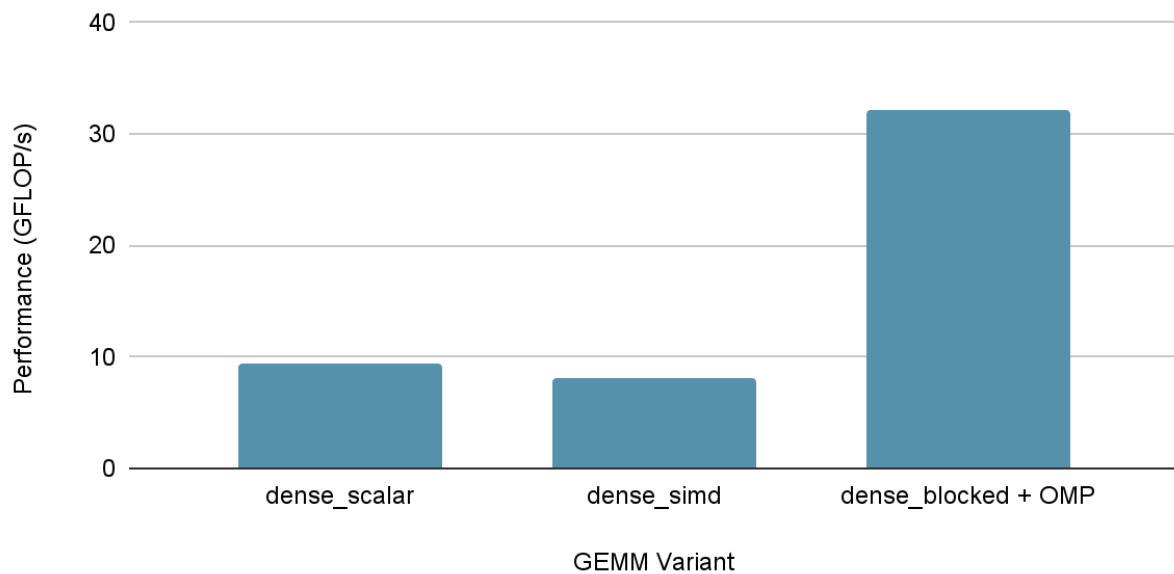
Both dense and sparse multiplications exactly matched the reference, validating functional correctness.

SIMD vs Scalar GEMM

Measured on 1024^3 multiplication:

Variant	Threads	Time (s)	GFLOP/s
dense_scalar	1	0.226612	9.4765
dense_simd	1	0.266479	8.0587
dense_blocked + OMP	4	0.0668402	32.1286

Dense GEMM: Scalar vs SIMD vs Blocked + Multithreaded Performance



In theory, SIMD vectorization should provide speedup over a scalar GEMM because multiple floating point operations are performed per instruction. However, in my results at the problem

size this was not the case. This may be due to when SIMD overhead outweighs the computational benefit when the tiles do not fit perfectly in the L1/L2 cache. The scalar version benefits from simple loops with excellent locality. As a result, in the experiments the multithreaded GEMM was the fastest overall while SIMD did not outperform scalar at this matrix size which is not expected but may be due to factors such as SIMD overhead effects on moderate problem sizes.

Thread Scaling Performance

Tested on 1024^3 , density 0.05.

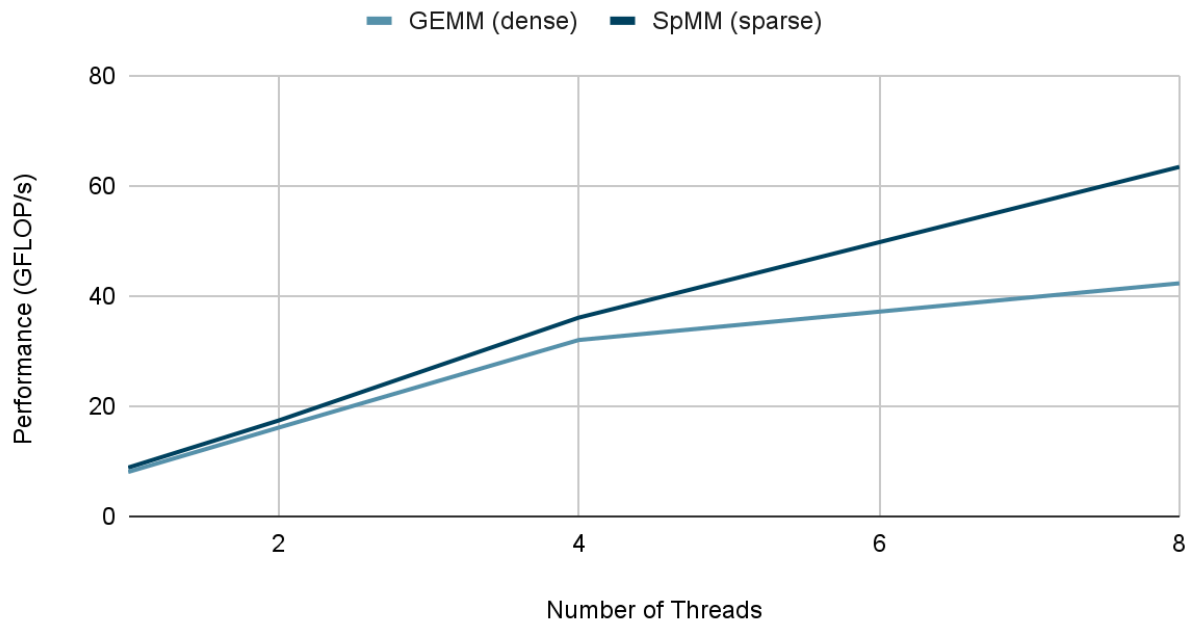
Dense GEMM Scaling

Threads	GFLOP/s
1	8.06
2	16.09
4	32.00
8	42.29

SpMM Scaling

Threads	GFLOP/s
1	8.85
2	17.38
4	36.06
8	63.41

Thread Scaling Performance for GEMM vs SpMM



Cycles Per Nonzero

Threads	Time (s)	nnz	CPNZ
1	0.0120447	52060	578
2	0.0061333	52060	589
4	0.0029565	52060	568
8	0.0016814	52060	646

The thread scaling results for both dense GEMM and SpMM show performance improvements as the number of threads increases, but with diminishing returns at higher parallelism levels. For dense GEMM, performance increases from 8 GFLOP/s at 1 thread to 42 GFLOP/s at 8 threads. This is expected as the kernel scales nearly linearly when moving from 1 to 4 threads. However, scaling from 4 to 8 threads provides a smaller relative gain because the computation begins to saturate the memory subsystem. SpMM shows a similar but more bandwidth sensitive scaling pattern. SpMM becomes bandwidth limited earlier than GEMM because of its inherently low arithmetic intensity and irregular memory access pattern which results in sub linear scaling at higher threads. The relatively constant CPNZ across threads indicates that each nonzero incurs

similar work while the increase at 8 threads reflects additional contention and memory bandwidth pressure.

Size Scaling

Measured at sizes:

128 → 256 → 512 → 1024 → 1536 → 2048 → 3072, density = 0.05, 4 threads

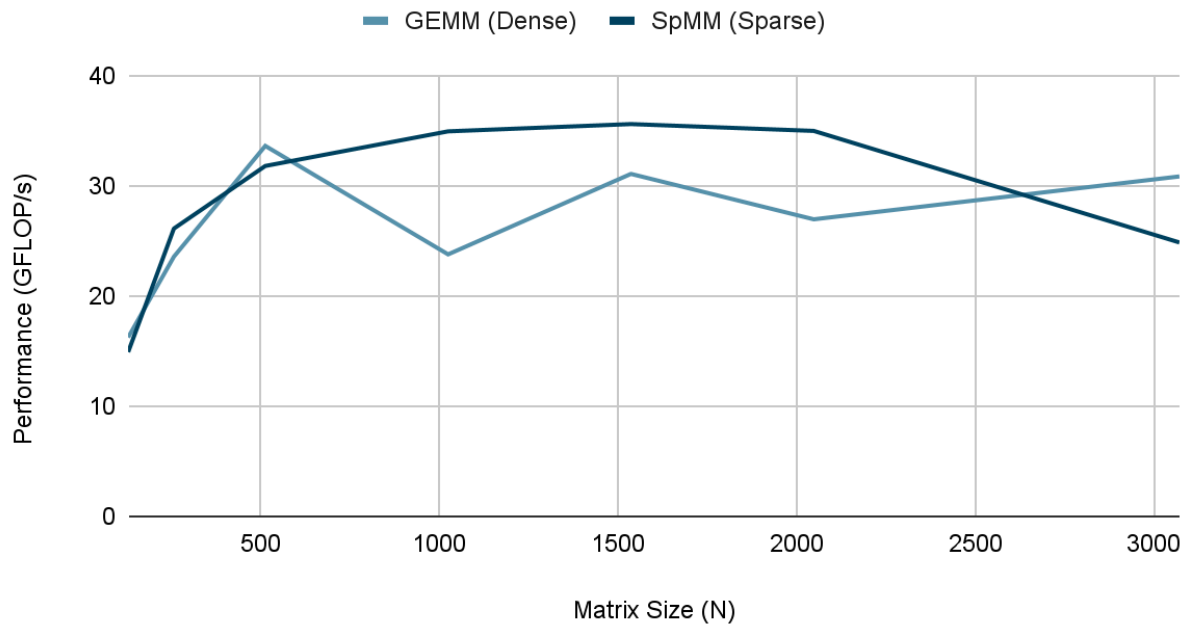
Dense GEMM Results

Size	GFLOP/s
128	16.24
256	23.56
512	33.61
1024	23.77
1536	31.07
2048	26.95
3072	30.84

SpMM Results

Size	GFLOP/s
128	14.91
256	26.11
512	31.79
1024	34.93
1536	35.59
2048	34.97
3072	24.85

Size Scaling Performance: GEMM vs SpMMored



Measured Effective Memory Bandwidth

Size	Kernel	Bytes (MB)	Time (s)	BW (GB/s)
1024	dense	12.58	0.0671	0.19
1024	spmm	13.00	0.0030	4.40
2048	dense	50.33	0.6376	0.08
2048	spmm	52.00	0.0245	2.12

The size scaling experiments show performance regions by the cache hierarchy and DRAM bandwidth of the hardware. For small problem sizes, dense GEMM operates at very high GFLOP/s because all tiles of matrices fit entirely within the L1 and L2 cache. As the size increases to 512 to 1024, performance begins to decline. Although tiling still provides good locality, the working set exceeds the capacity of the L2 cache and relies heavily on L3, introducing additional latency but keeping the computation mostly compute bound. At sizes 1024 to 3072 the tile working set no longer fits in the shared L3 cache. This forces evictions of matrix blocks and results in higher DRAM traffic. The sparse SpMM kernel shows similar behaviors but with weaker transitions. This is due to SpMM having low arithmetic intensity and irregular memory access patterns. SpMM is dominated by memory bandwidth across all sizes.

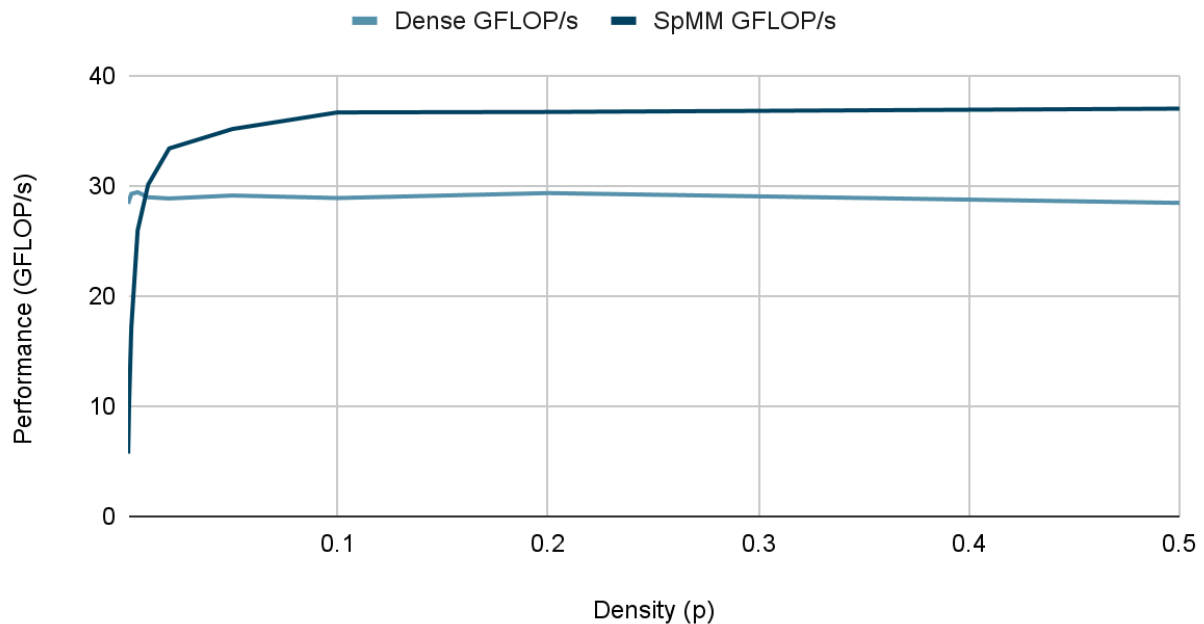
As a result, the performance grows with size and remains bandwidth limited, never entering a compute bound region. Overall, size scaling confirms the expectations being dense GEMM transitions through cache levels before becoming bandwidth bound at very large sizes, whereas sparse SpMM remains memory bound for all sizes due to structural sparsity and poor data reuse.

The measured bandwidth 2 to 4 GB/s for SpMM and <1 GB/s for dense GEMM is below the attainable 30 to 40 GB/s stream bandwidth on this CPU under native Linux. This confirms that SpMM is limited by irregular memory access and CSR indirection rather than raw DRAM throughput.

Density Sweep

Density p	Dense GFLOP/s	SpMM GFLOP/s	nnz
0.0005	28.36	5.69	2,048
0.0010	28.69	10.65	4,146
0.0020	29.26	17.15	8,389
0.0050	29.41	25.94	20,956
0.0100	28.95	30.10	41,822
0.0200	28.84	33.38	83,872
0.0500	29.11	35.14	209,148
0.1000	28.88	36.65	419,563
0.2000	29.33	36.69	838,078
0.5000	28.44	36.99	2,097,176

GEMM vs SpMM Performance Across Sparsity Density



The density sweep revealed that SpMM dramatically outperforms dense GEMM at low densities. This behavior is expected because SpMM processes only the nonzero elements, whereas dense GEMM always performs work regardless of sparsity. As density increases, SpMM runtime grows linearly, eventually approaching the cost of dense GEMM.

Roofline Analysis

Arithmetic Intensity

Dense GEMM

- $AI(1024^3) \approx 20 \text{ FLOP/byte}$
- $AI(2048^3) \approx 20 \text{ FLOP/byte}$

Compute bound

SpMM

(1024^3 , nnz=52060)

- $AI \approx 0.30 \text{ FLOP/byte}$

(2048³, nnz=209148)

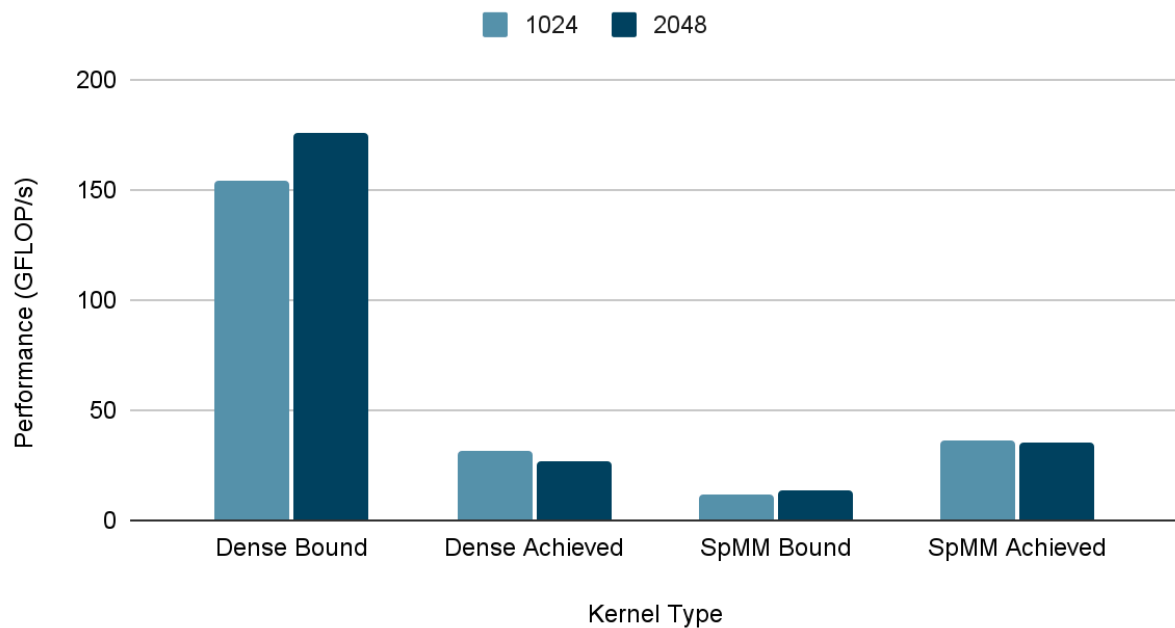
- AI $\approx$ 0.35 FLOP/byte

Memory bound

Roofline Table

Kernel	Size	Density	AI	Roofline Bound (GFLOP/s)	Achieved
dense	1024	N/A	20	154	32.0
spmm	1024	0.05	0.30	12	36.1
dense	2048	N/A	20	176	26.9
spmm	2048	0.05	0.35	14	35.0

Roofline Performance Bound for GEMM and SpMM



The roofline results separate the behavior of dense GEMM and sparse dense SpMM. Dense GEMM has high arithmetic intensity placing it into the compute bound region where performance is limited primarily by the CPU's floating point throughput rather than memory

bandwidth. SpMM at 5% density shows low arithmetic intensity which falls deep within the memory bound region.

From the size scaling experiments, SpMM achieved 2.1 to 4.4 GB/s of memory bandwidth depending on the problem size. This is below the theoretical WSL2 bandwidth limit (40 to 50 GB/s), confirming that SpMM is memory bound and limited primarily by irregular accesses and CSR indirection. The dense GEMM kernel achieves high compute utilization and is therefore compute-bound, consistent with its high arithmetic intensity (20 FLOP/byte).

Summary & Conclusions

This project implemented dense GEMM and sparse SpMM kernels and analyzed their performance using SIMD, blocking, and multithreading. Dense GEMM behaved as expected, showing strong scaling with threads and high performance at small sizes due to good cache locality. One unexpected result was that SIMD did not outperform the scalar GEMM at the tested size, which may be due to overhead. SpMM performance followed expected trends improving with increasing density and thread count but remained memory bound across all experiments because of low arithmetic intensity and irregular memory access. GEMM operated in the compute bound region while SpMM stayed bandwidth limited. Overall, the results aligned closely with theoretical behavior, with minor deviations explained by overheads.

References

- [1]OpenAI, “ChatGPT,” *chatgpt.com*, <https://chatgpt.com>
- [2]“Accelerating Matrix Multiplication with Block Sparse Format and NVIDIA Tensor Cores,” NVIDIA Technical Blog, Mar. 19, 2021.
<https://developer.nvidia.com/blog/accelerating-matrix-multiplication-with-block-sparse-format-and-nvidia-tensor-cores/>
- [3]“SparseZipper: Enhancing Matrix Extensions to Accelerate SpGEMM on CPUs,” Arxiv.org, 2016. <https://arxiv.org/html/2502.11353v1>
- [4]C. Ng, “Compute Bound & Memory Bound,” Medium, Jan. 07, 2025.
<https://levelup.gitconnected.com/compute-bound-memory-bound-90f0ddadaad4>
- [5]“Single Instruction, Multiple Data (SIMD),” celerdata.com.
<https://celerdata.com/glossary/single-instruction-multiple-data-simd>

